NF4 反量化算子实现与优化

一. 项目简介

开发一个 CUDA 程序，实现**单核**的 NF4 解量化算子，将压缩后的 4-bit 权重实时解压为 16-bit 浮点（BF16 或 FP16）输出。

二. 实现思路(naïve)

1. 线程映射策略：
   a) 每个线程处理 2 个元素（一对），对应 packed_weights 中的一个字节
   b) tid = 全局线程 ID，对应第 tid 个字节
   c) elem0 = tid * 2 是该线程负责的第一个元素索引
   d) Grid 大小 = ceil(total_elements / 2) / 256
2. 线程内处理流程：
   a) 从一个 uint8 中取出高 4 位和低 4 位，分别作为偶数位和奇数位元素的 NF4 查表索引。

```
uint8_t packed = packed_weights[tid];
uint8_t idx_even = (packed >> 4) & 0x0F;  // upper nibble -> even element (elem0)
uint8_t idx_odd  = packed & 0x0F;         // lower nibble -> odd element (elem0 + 1)
```

   b) 解第一个元素量化

```
int blk0  = static_cast<int>(elem0 / blocksize);
int grp0  = blk0 / group_size;
float sc0 = __half2float(code2[absmax_q[blk0]])
          * __half2float(absmax2[grp0])
          + offset;

half h0 = __float2half(c_nf4[idx_even] * sc0);
```

   c) 解第二个元素量化

```
if (elem0 + 1 < total_elements) {
    int blk1 = static_cast<int>((elem0 + 1) / blocksize);
    float sc1 = sc0;
    if (blk1 != blk0) {
        int grp1 = blk1 / group_size;
        sc1 = __half2float(code2[absmax_q[blk1]])
            * __half2float(absmax2[grp1])
            + offset;
    }
    half h1 = __float2half(c_nf4[idx_odd] * sc1);
```

   d) 向量化写出。

```
// Vectorized 32-bit store: two fp16 packed into one uint32_t
uint32_t packed_out = static_cast<uint32_t>(__half_as_ushort(h0))
                    | (static_cast<uint32_t>(__half_as_ushort(h1)) << 16);
reinterpret_cast<uint32_t*>(output)[tid] = packed_out;
```

三. 优化过程（计算资源为 1 张 A10 卡）

1. 第一次优化：

原始性能如下：

```
=== Input Info ===
Shape: 4096x4096  blocksize=64  group_size=256
Elements=16777216  packed_bytes=8388608  blocks=262144  groups=1024
Offset: 2.595265

=== Warming up (5 iterations) ===

=== Performance ===
My Kernel  : 0.1358 ms (avg of 20 iters) | Bandwidth: 310.73 GB/s
Ref Kernel : 0.0993 ms (avg of 20 iters) | Bandwidth: 424.93 GB/s
Speedup    : 0.73x
```

Ncu 分析：

关键指标	值
Compute (SM) Throughput	**74.42%**
Memory Throughput	36.44%
ALU Pipeline Utilization	**61.8%** （最高）
Issue Slots Busy	75.44%
IPC	3.02

结论：

NCU 明确指出：ALU is the highest-utilized pipeline (61.8%), executing integer and logic operations. 你的 kernel 是 compute-bound，不是 memory-bound。

GPU 上整数除法极其昂贵（编译器会展开为 ~20 条指令的乘法+移位序列）。

策略：整数除法 → 位移；将 preload 中的 a * b + offset 改为 __fmaf_rn(a, b, offset)

优化后性能：

```
=== Input Info ===
Shape: 4096x4096  blocksize=64  group_size=256
Elements=16777216  packed_bytes=8388608  blocks=262144  groups=1024
Offset: 2.595265

=== Warming up (5 iterations) ===

=== Performance ===
My Kernel  : 0.1070 ms (avg of 20 iters) | Bandwidth: 394.44 GB/s
Ref Kernel : 0.0993 ms (avg of 20 iters) | Bandwidth: 424.93 GB/s
Speedup    : 0.93x

DEBUG: first 100 outputs, 91 non-zero. raw[0..3]: 0x4040 0x3e24 0x3b7d 0xc040
```

提升 10%性能左右。

2. 第二次优化

优化前性能如下：

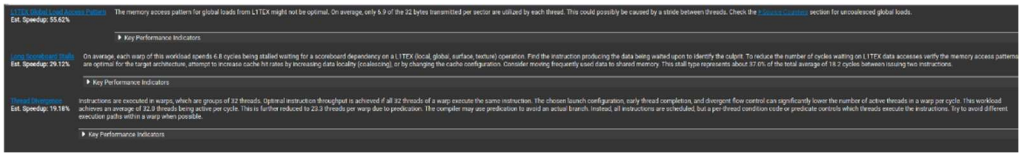
```
=== Input Info ===
Shape: 4096x4096  blocksize=64  group_size=256
Elements=16777216  packed_bytes=8388608  blocks=262144  groups=1024
Offset: 2.595265

=== Warming up (5 iterations) ===

=== Performance ===
My Kernel  : 0.1070 ms (avg of 20 iters) | Bandwidth: 394.44 GB/s
Ref Kernel : 0.0993 ms (avg of 20 iters) | Bandwidth: 424.93 GB/s
Speedup    : 0.93x

DEBUG: first 100 outputs, 91 non-zero. raw[0..3]: 0x4040 0x3e24 0x3b7d 0xc040
```

Ncu 分析：



结论：

三个瓶颈互相关联，核心根因推测是每线程工作量太少（仅 2 个元素）

策略：

增加向量化访存数量，**每线程处理 16-32 个元素**

优化后性能：

```
=== Input Info ===
Shape: 4096x4096  blocksize=64  group_size=256
Elements=16777216  packed_bytes=8388608  blocks=262144  groups=1024
Offset: 2.595265

=== Warming up (5 iterations) ===

=== Performance ===
My Kernel  : 0.1278 ms (avg of 20 iters) | Bandwidth: 330.28 GB/s
Ref Kernel : 0.0994 ms (avg of 20 iters) | Bandwidth: 424.71 GB/s
Speedup    : 0.78x

DEBUG: first 100 outputs, 91 non-zero. raw[0..3]: 0x4040 0x3e24 0x3b7d 0xc040
```

性能不升反降，code2[absmax_q[blk]] / absmax2[grp] 的散射访问和 L1TEX stall 仍然是主导原因。

四. 验证策略

1. 功能验证：

为验证功能正确性，编写了 python 程序。该程序定义了一组系统性的测试用例 test_cases，覆盖：

标准对齐尺寸（例如 64x64, 128x256, 512x1024）；

非对齐、奇数尺寸（100x100, 127x255, 33x77 等），专门检查边界处理和索引逻辑的鲁棒性；

不同 blocksize（64/128）以验证对分块参数的适配性。使用 bitsandbytes 官方库的 quantize_4bit / dequantize_4bit 作为 NF4 双重量化/反量化的参考实现，验证了所写算子的功能正确性。

2. 性能验证：

本项目中，在 Python 侧（test_correctness.py），加速比是以 bitsandbytes 官方 Python API 作为基线，采用端到端测量方式计算的：脚本先从可执行程序输出中解析我们 CUDA 核函数的平均时间 my_kernel_ms，再使用 CUDA events 在 Python 中多次调用 F.dequantize_4bit(packed, state) 得到 bitsandbytes 的平均时间 bnb_python_ms，最后计算加速比。这个指标包含了 Python 调度、API 封装等高层开销，因此偏向应用层、端到端意义的性能评估。

相对地，在 CUDA C++ 可执行程序（src/nf4_dequantize.cu）中，我单独实现了一套参考路径 bnb_ref_dequant_absmax + bnb_ref_dequant_nf4，严格模仿 bitsandbytes 的 kDequantizeBlockwise 算法，并在同一进程、同一数据布局下与目标核函数一起用 CUDA events 在设备端计时：先得到核函数的平均时间 avg_ms，再得到参考实现的平均时间 ref_avg_ms，然

给出一个纯 GPU 侧、内核级别的加速比。之所以不在 C++ 中直接调用 bitsandbytes 的 Python API，而是写显式的参考 kernel，有几方面原因：一是避免嵌入 Python 解释器与跨语言调用带来的额外开销和抖动，保证计时仅反映设备端计算本身，从而满足微基准测试和 roofline 分析对“干净计时”的要求；二是保证两条路径的数据传输、内存分配、流与事件管理完全对称，可在 Nsight Compute 中在同一可执行文件内并列分析两个 kernel 的访存模式和流水线行为，形成严格的“同场对比”；三是可精确控制参考实现的细节，使其与 bitsandbytes 的实际算法结构一一对应，同时又不受 Python 包版本、运行时环境等外部因素影响。

五. 性能总结与分析

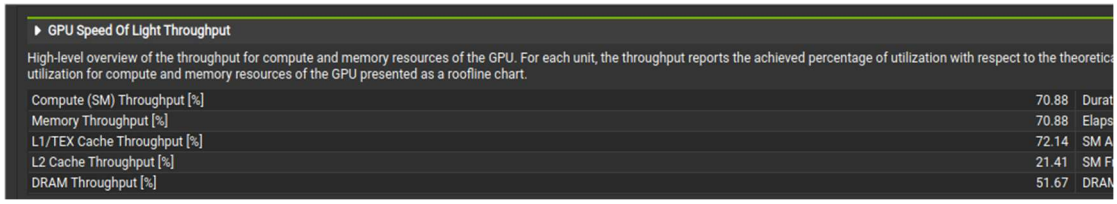
最终性能如下:

```
=== Input Info ===
Shape: 4096x4096  blocksize=64  group_size=256
Elements=16777216  packed_bytes=8388608  blocks=262144  groups=1024
Offset: 2.595265

=== Warming up (5 iterations) ===

=== Performance ===
My Kernel  : 0.1070 ms (avg of 20 iters) | Bandwidth: 394.44 GB/s
Ref Kernel : 0.0993 ms (avg of 20 iters) | Bandwidth: 424.93 GB/s
Speedup    : 0.93x
```

后续尝试多种优化思路，仍未有效提升性能，从 ncu 分析的结果看，目前的算子已经处于一个“算 / 访存都用得比较满”的平衡状态。



GPU Speed Of Light Throughput

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretica
utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	70.88	Durat
Memory Throughput [%]	70.88	Elaps
L1/TEX Cache Throughput [%]	72.14	SM A
L2 Cache Throughput [%]	21.41	SM F
DRAM Throughput [%]	51.67	DRAM

在不牺牲可维护性、也不改变输入格式的前提下，进一步优化会越来越像“抠细节”。

如果希望再拿到 20–30% 级别的提升，通常需要：

增加 per-thread 工作量；改变整体调度方式（例如用 persistent kernel / tile-based 方案，让一个 CTA 反复处理多个 block / group，充分 amortize 预加载成本）。

这两类改动更接近“重新设计 kernel”。